Free Pascal

Сучасна модель конкурентності, асинхронного I/O та узагальнень

Професійний посібник для користувача-розробника з описом нововведень

Цільова версія:	Free Pascal 3.3.1 (Trunk)
Робоча гілка:	`feature/concurrency`
Охоплені етапи:	Фази 1.1, 1.2, 1.3, 1.4 та Розділ 2.1
Підтримувані ОС:	Linux (`io_uring`, `epoll`), macOS / BSD (`kqueue`), Windows (`IOCP`)
Архітектура CPU:	x86_64 (AMD64 System V ABI / Win64) та AArch64

Ключові архітектурні зміни гілки

1. **Синтаксис мови:** `{$modeswitch implicitgenerics}` дозволяє повністю відкинути архаїчне ключове слово `specialize` та вмикає автоматичне виведення типів для узагальнених методів і анонімних функцій.
2. **Оркестрація задач:** `System.Threading` з неблокувальними продовженнями (`.ThenBy`, `.ContinueWith`) та токенами кооперативного скасування (`TCancellationToken`).
3. **Потокобезпечна комунікація:** Модель каналів CSP (`TChannel<T>`) у стилі мови Go та ультрашвидкий `TSlimReaderWriterLock` із RAII-гардами.
4. **Реактивний I/O:** Пакет `fcl-eventloop` на нативних мультиплексорах ядра ОС для обслуговування 100 000+ з'єднань.
5. **Корутини:** Пакет `fcl-async` (`TFiber`, генератори) та підтримка `async`/`await` на рівні компілятора.

2026 рік

Contents

1 Архітектурна модель та вибір парадигми **2**

2 Синтаксична ергономіка: {$modeswitch implicitgenerics} **3**
2.1 Опущення ключового слова specialize 3
2.2 Автоматичне виведення типів (Generic Type Inference) 3

3 Задачі, конвеєри та скасування (System.Threading) **5**
3.1 Кооперативне скасування: TCancellationTokenSource 5
3.2 Неблокувальні ланцюжки задач: .ThenBy та .ContinueWith 5

4 Нові примітиви синхронізації (SyncObjs) **7**
4.1 Канали CSP: generic TChannel<T> 7
4.2 TSlimReaderWriterLock та RAII-гарди 7

5 Масштабований асинхронний I/O (fcl-eventloop) **9**
5.1 Побудова сервера на базі IEventLoop 9
5.2 TEventLoopGroup: Multi-Reactor на 100 000+ з'єднань 9

6 Корутини, файбери та async/await (fcl-async) **10**
6.1 Легковагові стекові файбери: TFiber 10
6.2 Ледачі послідовності: generic TGenerator<T> 10

7 Комплексний наскрізний приклад **12**

8 Правила надійності та перевірка під Valgrind **14**

§1. Архітектурна модель та вибір парадигми

Сучасний Free Pascal у гілці feature/concurrency надає розробнику цілісний трирівневий набір інструментів, що усуває потребу в прямій низькорівневій роботі з системними потоками (TThread) та блокуючими м'ютексами.

Клас задачі	Основний модуль	Ключові класи/типи	Переваги
CPU-Bound (обчислення, стиснення, парсинг)	System.Threading	TParallel.For, ITask, IFuture<T>, ThenBy	Автоматичне балансування навантаження (*work stealing*), мінімум блокувань.
Синхронізація (обмін повідомленнями, буферизація)	SyncObjs	TChannel<T>, TSlimReaderWriterLock, TBarrier	CSP-комунікація, усунення гонок даних, відсутність дескрипторів ядра ОС.
I/O-Bound (High-Load мережа, сокети, таймери)	fcl-eventloop	IEventLoop, TEventLoopGroup, IAsyncSocket	Нативні мультиплексори (io_uring, epoll), 100 000+ з'єднань на групу потоків.
Корутини (лінійний код для асинхронних операцій)	fcl-async та компілятор	TFiber, TGenerator<T>, async / await	Стек файбера перемикається за $<$10 нс без переривання системного потоку ОС.

Table 1: Матриця підбору парадигми конкурентності

§2. Синтаксична ергономіка: {$modeswitch implicitgenerics}

2.1 Опущення ключового слова specialize

Раніше в режимі {$mode objfpc} використання будь-якого узагальненого типу або виклику методу вимагало громіздкої конструкції specialize.

Перемикач {$modeswitch implicitgenerics} дозволяє використовувати звичний, лаконічний синтаксис для **всіх** типів (як нових, так і стандартних FCL):

```
{$mode objfpc}{$H+}
{$modeswitch implicitgenerics}

uses fgl, Generics.Collections, System.Threading;

type
  { Оголошення типів без specialize }
  TIntList   = TFPGList<Integer>;
  TUserCache = TDictionary<string, Int64>;

var
  List: TFPGList<Integer>;
  Map:  TDictionary<string, Int64>;
  Fut:  IFuture<string>;
begin
  List := TFPGList<Integer>.Create;
  Map  := TDictionary<string, Int64>.Create;

  { Явний виклик generic-методу без specialize }
  Fut := TTask.Future<string>(
    function: string
    begin
      Result := 'Дані готові';
    end);
end.
```

2.2 Автоматичне виведення типів (Generic Type Inference)

Компілятор автоматично визначає параметри <T> із переданих фактичних аргументів. Це стосується чисел, рядків, інтерфейсів, об’єктів та, що найважливіше, **типів повернення анонімних функцій**:

```
var
  FutStr: IFuture<string>;
  FutInt: IFuture<Integer>;
begin
  // T = string виводиться автоматично з типу повернення:
  FutStr := TTask.Future(
    function: string
    begin
      Result := 'Привіт, світ';
    end);

  // T = Integer виводиться автоматично:
  FutInt := TTask.Future(
    function: Integer
    begin
```

```
16      Result := 42;
17    end);
18 end.
```

Працює для всіх функцій і методів

Type Inference не обмежується лише тасками. Будь-яка узагальнена функція на зразок `generic function MaxVal<T>(A, B: T): T` тепер викликається просто: `M := MaxVal(3.14, 2.71);` без вказування `<Double>`.

§3. Задачі, конвеєри та скасування (System.Threading)

3.1 Кооперативне скасування: TCancellationTokenSource

Джерело токенів скасування (TCancellationTokenSource) генерує легковагові структури TCancellationToken, які передаються в операції для їх узгодженого переривання:

```
var
  CTS: TCancellationTokenSource;
  Token: TCancellationToken;
  Task: ITask;
begin
  CTS := TCancellationTokenSource.Create;
  Token := CTS.Token;

  Task := TTask.Run(
    procedure
    begin
      while not Token.IsCancellationRequested do
      begin
        DoWorkBatch;
      end;
      Token.ThrowIfCancellationRequested; // Генерує EOperationCancelled
    end, Token);

  // Скасування операції з іншого потоку або за таймаутом:
  CTS.CancelAfter(500); // скасувати за 500 мс
end.
```

3.2 Неблокувальні ланцюжки задач: .ThenBy та .ContinueWith

Замість блокування потоку у виклику .Wait або .Value, використовується побудова неблокувальних конвеєрів:

```
var
  DownloadTask: IFuture<string>;
  ParseTask:    IFuture<Integer>;
begin
  // 1. Початкова задача
  DownloadTask := TTask.Future(
    function: string
    begin
      Result := HttpGet('https://api.service.internal/count');
    end);

  // 2. Неблокувальне продовження ThenBy (TResult = Integer виведено автоматично)
  ParseTask := DownloadTask.ThenBy(
    function(const Prev: ITask): Integer
    begin
      Result := StrToIntDef(IFuture<string>(Prev).Value, 0);
    end);

  // 3. Фінальна обробка результату або винятку
  ParseTask.ContinueWith(
    procedure(const Completed: ITask)
    begin
```

```
23        if Completed.Status = TTaskStatus.Completed then
24          WriteLn('Кількість: ', IFuture<Integer>(Completed).Value)
25        else
26          WriteLn('Помилка: ', Completed.ExceptionObject.Message);
27      end);
28  end.
```

§4. Нові примітиви синхронізації (SyncObjs)

4.1 Канали CSP: generic TChannel<T>

Канал реалізує модель Communicating Sequential Processes (як у мові Go). Він підтримує як обмежену буферизацію (Capacity > 0 з автоматичним блокуванням записувачів при заповненні), так і необмежену.

```
var
  Channel: TChannel<string>;
begin
  Channel := TChannel<string>.Create(50); // Буфер на 50 повідомлень

  // Письменник:
  TTask.Run(
    procedure
    var I: Integer;
    begin
      for I := 1 to 1000 do
        Channel.Send('Повідомлення #' + IntToStr(I)); // Backpressure
      Channel.Close;
    end);

  // Читач з прямим обходом for..in без накладних алокацій:
  TTask.Run(
    procedure
    var Item: string;
    begin
      for Item in Channel do
      begin
        ProcessItem(Item);
      end;
      WriteLn('Канал закрито, обробку завершено.');
    end);
end.
```

4.2 TSlimReaderWriterLock та RAII-гарди

Надвисокопродуктивне блокування читання/запису без системних дескрипторів ядра ОС. Підтримує пріоритет записувачів, щоб запобігти голодуванню, та RAII-гарди:

```
var
  Lock: TSlimReaderWriterLock;

procedure ReadResource;
var
  Guard: TSlimReaderWriterLock.IReadLock;
begin
  Guard := Lock.ReadLock; // Захоплює дозвіл на читання
  // Ресурс захищено. Звільнення відбувається автоматично при виході зі скоупу!
end;

procedure WriteResource;
var
  Guard: TSlimReaderWriterLock.IWriteLock;
begin
```

```
16    Guard := Lock.WriteLock; // Ексклюзивний дозвіл на запис
17    UpdateResourceData;
18  end;
```

§5. Масштабований асинхронний I/O (fcl-eventloop)

Пакет fcl-eventloop реалізує патерн Reactor поверх найбільш ефективних нативних API операційної системи:

- **Linux:** io_uring (для ядер 5.1+) та високошвидкісний epoll;
- **macOS / FreeBSD:** kqueue;
- **Windows:** IOCP (I/O Completion Ports).

5.1 Побудова сервера на базі IEventLoop

```
var
  Loop:   IEventLoop;
  Server: IAsyncServerSocket;
begin
  Loop := TEventLoop.Create(bkAuto); // Автоматичний вибір кращого бекенду
  Server := TEventLoop.CreateTcpServer(Loop);
  Server.Listen('0.0.0.0', 8080);

  Server.AsyncAccept(
    procedure(AHandle: TLoopHandle; const Addr: string; Port: Word; Status: Integer)
    var
      Client: IAsyncSocket;
      Buffer: array[0..4095] of Byte;
    begin
      if Status <> 0 then Exit;
      Client := TAsyncSocket.Create(Loop, AHandle, True);

      Client.AsyncRead(@Buffer[0], SizeOf(Buffer),
        procedure(Bytes: NativeInt; ReadStatus: Integer)
        begin
          if (ReadStatus = 0) and (Bytes > 0) then
          begin
            // Неблокувальне відлуння (Echo)
            Client.AsyncWrite(@Buffer[0], Bytes, nil);
          end
          else
            Client.Close;
        end);
    end);

  Loop.Run;
end.
```

5.2 TEventLoopGroup: Multi-Reactor на 100 000+ з'єднань

Клас TEventLoopGroup створює пул незалежних потоків-реакторів (наприклад, по одному на кожне апаратне ядро процесора). Нові з'єднання балансуються між реакторами методом Round-Robin або Least-Connections, усуваючи конкуренцію за чергу подій.

§6. Корутини, файбери та async/await (fcl-async)

6.1 Легковагові стекові файбери: TFiber

TFiber дозволяє писати лінійний код, який призупиняється в очікуванні асинхронних операцій без блокування системного потоку ОС.

- Контекст перемикається на вбудованому асемблері AMD64 менш ніж за 10 нс;
- Гарантується 16-байтне вирівнювання стеку під SIMD/AVX інструкції;
- Стек винятків FPC (ExceptAddrStack) симетрично ізолюється для кожного файбера.

```
var
  WorkerFiber: TFiber;
begin
  WorkerFiber := TFiber.Create(
    procedure
    var
      Fut: IFuture<string>;
      Data: string;
    begin
      WriteLn('Файбер розпочав виконання');

      Fut := TTask.Future(
        function: string
        begin
          Sleep(200);
          Result := 'Завантажені дані';
        end);

      // Призупиняє ВИКЛЮЧНО поточний файбер, потік ОС продовжує іншу роботу!
      Data := TFiber.Await(Fut);

      WriteLn('Файбер відновився з результатом: ', Data);
    end);

  WorkerFiber.Start;
end.
```

6.2 Ледачі послідовності: generic TGenerator<T>

Генератори реалізують патерн ітератора, де кожне наступне значення обчислюється лише тоді, коли його запитує цикл for..in:

```
function GenerateFibonacci(Limit: Integer): TGenerator<Int64>;
begin
  Result := TGenerator<Int64>.Create(
    procedure(const Yield: TGenerator<Int64>.TYieldProc)
    var A, B, C: Int64;
    begin
      A := 0; B := 1;
      while A <= Limit do
      begin
        Yield(A); // Призупиняє генератор і повертає значення у for..in
        C := A + B; A := B; B := C;
```

```
      end;
    end);
end;

var Num: Int64;
begin
  for Num in GenerateFibonacci(1000) do
    WriteLn(Num);
end.
```

§7. Комплексний наскрізний приклад

Наведений нижче приклад демонструє спільну роботу всіх компонентів: високонавантажений TCP-сервер на fcl-eventloop приймає байти, через TChannel<string> передає їх у пул потоків System.Threading, де вони паралельно трансформуються через ланцюжок .ThenBy з автоматичним виведенням типів:

```
program EndToEndModernConcurrency;
{$mode objfpc}{$H+}
{$modeswitch implicitgenerics}
{$modeswitch anonymousfunctions}
{$modeswitch functionreferences}

uses
  {$IFDEF UNIX}cthreads,{$ENDIF}
  SysUtils, SyncObjs, System.Threading,
  eventloop.types, eventloop.core, eventloop.sockets, fcleventloop;

var
  WorkQueue: TChannel<string>;
  Loop:      IEventLoop;
  Server:    IAsyncServerSocket;
  CTS:       TCancellationTokenSource;

begin
  CTS := TCancellationTokenSource.Create;
  WorkQueue := TChannel<string>.Create(1000); // Буфер із backpressure

  // 1. Робочий потікдиспетчер- пулу задач:
  TTask.Run(
    procedure
    var Msg: string;
    begin
      for Msg in WorkQueue do
      begin
        // Обчислення в пулі потоків з автоматичним виведенням типів
        TTask.Future(
          function: Integer
          begin
            Result := Length(Msg);
          end
        ).ThenBy(
          function(const Prev: ITask): string
          begin
            Result := 'Довжина вхідного пакета: ' +
    IntToStr(IFuture<Integer>(Prev).Value);
          end
        ).ContinueWith(
          procedure(const T: ITask)
          begin
            WriteLn(IFuture<string>(T).Value);
          end);
      end;
    end, CTS.Token);

  // 2. Реактор мережевого I/O (fcl-eventloop):
  Loop := TEventLoop.Create(bkAuto);
  Server := TEventLoop.CreateTcpServer(Loop);
```

```
  Server.Listen('0.0.0.0', 9000);

  Server.AsyncAccept(
    procedure(AHandle: TLoopHandle; const Addr: string; Port: Word; Status: Integer)
    var
      Sock: IAsyncSocket;
      Buf: array[0..2047] of AnsiChar;
    begin
      if Status <> 0 then Exit;
      Sock := TAsyncSocket.Create(Loop, AHandle, True);
      Sock.AsyncRead(@Buf[0], SizeOf(Buf),
        procedure(Bytes: NativeInt; ReadStatus: Integer)
        begin
          if (ReadStatus = 0) and (Bytes > 0) then
            WorkQueue.Send(Copy(Buf, 1, Bytes))
          else
            Sock.Close;
        end);
    end);

  WriteLn('Сервер запущено на порті 9000. Очікування клієнтів...');
  Loop.Run;

  WorkQueue.Close;
  CTS.Cancel;
  CTS.Free;
  WorkQueue.Free;
end.
```

§8. Правила надійності та перевірка під Valgrind

1. **Ніколи не блокуйте потік реактора `IEventLoop`:** Будь-які блокуючі системні виклики (робота з диском, звернення до СУБД, важкі обчислення) повинні делегуватися у пул потоків через `TTask.Run` або передаватися в чергу `TChannel<T>`. Потік `EventLoop` повинен лише мультиплексувати дескриптори.
2. **Ініціалізація `cthreads` на Linux/Unix:** Будь-яка багатопотокова програма зобов'язана першим модулем у секції `uses` підключати модуль `cthreads`, інакше внутрішні примітиви синхронізації RTL працюватимуть некоректно.
3. **Перевірка детекторами гонок даних (Helgrind / DRD):** Усі нові примітиви (`TCancellationToken`, `TSlimReaderWriterLock`, `TChannel`, `TCountdownEvent`, `TBarrier`) підтримують ключ компілятора `-dFPC_VALGRIND`. У бінарний код вбудовуються клієнтські запити Valgrind (*happens-before / happens-after*), що гарантує повну відсутність хибних спрацювань під час аналізу.

Корисні посилання в репозиторії

- `docs/concurrency/modern-concurrency-reference.md` — повний довідник API;
- `docs/concurrency/guide-async.md` — файбери та корутини;
- `docs/concurrency/guide-eventloop.md` — цикл подій та архітектура реактора;
- `docs/concurrency/roadmap-improvements.md` — дорожня карта гілки `feature/concurrency`.